Guía de uso · versión `ml`

Paso a paso para usar `franki678/bigdata-sandbox:ml`: el mismo cluster de la versión `base` más Python para datos (pandas, scikit-learn, XGBoost, matplotlib, pymongo), Spark MLlib y JupyterLab. El recorrido va de datos sucios a modelos: limpiar con pandas, lo mismo con Spark, entrenar con scikit-learn, lo mismo con Spark MLlib, leer MongoDB con pandas y, al final, el estudio de ML (`sandbox_ml`) desde un notebook.

README de `ml` · Mapa interactivo · Guía de `base` · Glosario · README del proyecto

Todo lo de la guía de `base` (HDFS, MapReduce, Pig, Hive, Sqoop, Spark, MongoDB, HA) anda igual en `ml`: esta guía cubre sólo lo que se agrega. Los resultados que aparecen (97200 tickets limpios, 20000 clientes…) salen de los generadores con semilla fija: a todos les da lo mismo.

Contenido

1. Antes de empezar
2. Crear el cluster con la imagen ml
3. JupyterLab
4. Datos sucios con pandas
5. El mismo ETL con Spark
6. Modelos con scikit-learn
7. Los mismos modelos con Spark MLlib
8. De MongoDB a pandas
9. El estudio de ML desde JupyterLab
10. Problemas comunes y limpieza

---

1. Antes de empezar

- Docker instalado y andando: guía de `base`, parte 2.
- **7 GB de RAM para Docker** (medido por el workflow `Imagen`: en reposo 2,2 GB, pico 6,0 GB con 2 DataNodes). Descarga 1,58 GB; en disco, 2,93 GB en Linux y ~4,5 GB en Docker Desktop.
- No hay archivo del aula para `ml` (el release `imagen` es sólo `base`): se baja de Docker Hub.

Las dos terminales siguen igual: `docker ...` en **tu compu**, el resto **adentro** (`docker exec -it bigdata bash`). Y ahora hay una tercera forma de entrar: **JupyterLab**, en el navegador.

---

2. Crear el cluster con la imagen ml

```
docker pull franki678/bigdata-sandbox:ml
```

Si ya tenías el contenedor de `base`, se borra (el volumen y tus datos quedan; la imagen `ml` usa el mismo volumen sin problema):

```
docker rm -f bigdata
```

El mismo `docker run` de `base`, con un puerto más (8888, JupyterLab) y la imagen `:ml`. Mac y Linux:

```
docker run -d --name bigdata --stop-timeout 20 \
  -e DATANODES=3 -e REPLICATION=2 -e BLOCK_SIZE=1m -e MONGODB=si \
  -p 127.0.0.1:9870:9870 -p 127.0.0.1:9868:9868 -p 127.0.0.1:9901-9909:9901-9909 \
  -p 127.0.0.1:8088:8088 -p 127.0.0.1:8042:8042 -p 127.0.0.1:19888:19888 \
  -p 127.0.0.1:10000:10000 -p 127.0.0.1:10002:10002 \
  -p 127.0.0.1:18080:18080 -p 127.0.0.1:4040:4040 \
  -p 127.0.0.1:3306:3306 -p 127.0.0.1:27017:27017 -p 127.0.0.1:8888:8888 \
  -v bigdata_hadoop:/hadoop -v "$(pwd)/datos:/datos" \
  franki678/bigdata-sandbox:ml
```

Windows, PowerShell:

```
docker run -d --name bigdata --stop-timeout 20 -e DATANODES=3 -e REPLICATION=2 -e BLOCK_SIZE=1m -e MONGODB=si -p 127.0.0.1:9870:9870 -p 127.0.0.1:9868:9868 -p 127.0.0.1:9901-9909:9901-9909 -p 127.0.0.1:8088:8088 -p 127.0.0.1:8042:8042 -p 127.0.0.1:19888:19888 -p 127.0.0.1:10000:10000 -p 127.0.0.1:10002:10002 -p 127.0.0.1:18080:18080 -p 127.0.0.1:4040:4040 -p 127.0.0.1:3306:3306 -p 127.0.0.1:27017:27017 -p 127.0.0.1:8888:8888 -v bigdata_hadoop:/hadoop -v "${PWD}/datos:/datos" franki678/bigdata-sandbox:ml
```

Windows, CMD:

```
docker run -d --name bigdata --stop-timeout 20 -e DATANODES=3 -e REPLICATION=2 -e BLOCK_SIZE=1m -e MONGODB=si -p 127.0.0.1:9870:9870 -p 127.0.0.1:9868:9868 -p 127.0.0.1:9901-9909:9901-9909 -p 127.0.0.1:8088:8088 -p 127.0.0.1:8042:8042 -p 127.0.0.1:19888:19888 -p 127.0.0.1:10000:10000 -p 127.0.0.1:10002:10002 -p 127.0.0.1:18080:18080 -p 127.0.0.1:4040:4040 -p 127.0.0.1:3306:3306 -p 127.0.0.1:27017:27017 -p 127.0.0.1:8888:8888 -v bigdata_hadoop:/hadoop -v "%cd%\datos:/datos" franki678/bigdata-sandbox:ml
```

Con docker compose: en el `docker-compose.yml` del repo, cambiar la imagen a `franki678/bigdata-sandbox:ml` y descomentar el puerto `8888`.

Los parámetros son los de `base` (guía de `base`, parte 4) más uno:

Parámetro	Qué elige	Por defecto	Valores
`JUPYTER`	Prende JupyterLab en el 8888	`si`	`si`, `no`

Esperá el **Cluster LISTO**:

```
docker logs -f bigdata
```

`estado` ahora muestra JupyterLab y dice que la imagen es la `ml`:

```
docker exec bigdata estado
```

Adentro (`docker exec -it bigdata bash`), los ejemplos de esta guía están en `ejemplos/ml`:

```
ls ejemplos/ml
```

Las librerías, con versiones fijas (todos tienen exactamente lo mismo):

```
python3 -c "import numpy, pandas, sklearn, pyspark; print('numpy', numpy.__version__);
print('pandas', pandas.__version__); print('scikit-learn', sklearn.__version__);
print('pyspark', pyspark.__version__)"
```

Y JupyterLab contesta:

```
curl -s http://localhost:8888/api
```

---

3. JupyterLab

Abrí **http://localhost:8888** en el navegador. No pide contraseña ni token (sólo se ve desde tu compu, por el `127.0.0.1`). A la izquierda están los archivos de `/datos`, que es tu carpeta `datos/`: un notebook que guardes ahí queda en tu compu.

- **Nuevo notebook:** *File → New → Notebook → Python 3 (ipykernel)*. Se escribe en celdas y cada una se corre con `Shift+Enter`. Lo último de la celda se muestra solo.
- **Terminal:** *File → New → Terminal* abre una consola bash adentro del contenedor, igual que `docker exec -it bigdata bash`.
- Si cerrás la pestaña, el notebook sigue vivo; *Kernel → Shut Down Kernel* lo apaga y libera la memoria.

Probá en un notebook nuevo, una celda por bloque:

```
import numpy as np
import pandas as pd

tabla = pd.DataFrame({"producto": ["yerba", "mate", "termo"],
                      "precio": [2500, 8000, 35000]})
tabla["con_iva"] = np.round(tabla["precio"] * 1.21)
tabla
```

```
import matplotlib.pyplot as plt

tabla.plot(x="producto", y="precio", kind="bar", legend=False)
plt.ylabel("pesos")
plt.show()
```

Spark también anda desde el notebook (`import pyspark` funciona en cualquier `python3` de la imagen). La sesión corre sobre YARN, como con `spark-submit`:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("primer notebook").getOrCreate()
print(spark.sparkContext.master, spark.version)
spark.range(1000).selectExpr("sum(id)").show()
```

Una sesión de Spark abierta ocupa memoria de YARN aunque no haga nada. Al terminar:

```
spark.stop()
```

---

4. Datos sucios con pandas

ETL es **E**xtraer (leer los datos como vienen), **T**ransformar (limpiarlos) y **C**argar (guardarlos listos para analizar). `generar_sucios.py` arma 100.000 ventas y les mete a propósito, en cantidades conocidas, la suciedad típica:

Problema	Cuánto
Tickets duplicados	3 %
Fechas en tres formatos	`2026-03-05`, `05/03/2026` y `20260305`
Texto desprolijo	mayúsculas, espacios y tildes (`" YERBA"`, `"Débito "`)
Precios con formato argentino	15 % (`"2.575,00"`)
Nulos	cantidad 2 %, precio 1 %, medio de pago 5 %
Errores de tipeo	cantidad ×100 (0,5 %), cantidad negativa (0,3 %), precio sin coma (0,5 %)

Los precios suben 3 % por mes (inflación). Al final dice cuántas filas tendrían que quedar:

```
python3 ejemplos/ml/generar_sucios.py
ls -l ventas_sucias.csv
```

Mirar antes de limpiar

En un notebook. `dtype=str` lee todo como texto, para ver los datos tal como vienen:

```
import pandas as pd

sucias = pd.read_csv("ventas_sucias.csv", dtype=str)
print(sucias.shape)
sucias.head(8)
```

```
sucias.isna().sum()
```

```
print("tickets repetidos:", sucias["ticket"].duplicated().sum())
sucias["producto"].value_counts().head(12)
```

```
sucias["fecha"].str.len().value_counts()
```

Limpiar

`etl_pandas.py` hace la limpieza paso a paso y cuenta lo que arregla en cada uno, para compararlo con lo que dijo el generador. Deja `ventas_limpias.parquet`:

```
python3 ejemplos/ml/etl_pandas.py
```

Los pasos, en orden: sacar duplicados, unificar el texto (recortar, minúsculas, sin tildes), leer las tres formas de fecha, pasar los precios argentinos a número, marcar los errores de tipeo, completar o descartar los nulos, y calcular el `importe`. Por ejemplo, las fechas (fragmento del script):

```
f = ventas["fecha"]
iso = f.str.fullmatch(r"\d{4}-\d{2}-\d{2}", na=False)        # 2026-03-05
argentina = f.str.fullmatch(r"\d{2}/\d{2}/\d{4}", na=False)  # 05/03/2026
junta = f.str.fullmatch(r"\d{8}", na=False)                  # 20260305
ventas["fecha"] = (pd.to_datetime(f.where(iso), format="%Y-%m-%d")
                   .fillna(pd.to_datetime(f.where(argentina), format="%d/%m/%Y"))
                   .fillna(pd.to_datetime(f.where(junta), format="%Y%m%d")))
```

Mirar después

```
limpias = pd.read_parquet("ventas_limpias.parquet")
print(limpias.dtypes)
limpias[limpias["ticket"].isin([5, 76331])]
```

La inflación se ve en la mediana del precio por mes:

```
yerba = limpias[limpias["producto"] == "yerba"]
yerba.groupby(yerba["fecha"].dt.month)["precio"].median()
```

```
import matplotlib.pyplot as plt

print(limpias.groupby("producto")["importe"].sum().sort_values(ascending=False))
limpias.groupby(limpias["fecha"].dt.month)["importe"].sum().plot(kind="bar")
plt.xlabel("mes")
plt.ylabel("facturado (pesos)")
plt.show()
```

---

5. El mismo ETL con Spark

pandas trabaja en la memoria de una compu; Spark reparte el trabajo en el cluster. `etl_spark.py` hace **exactamente** los mismos pasos que `etl_pandas.py`, leyendo de HDFS, y al final imprime el mismo resumen: tienen que dar igual número por número.

```
hdfs dfs -mkdir -p /user/root/ml
hdfs dfs -put -f ventas_sucias.csv /user/root/ml/
hdfs dfs -ls /user/root/ml
```

`2> etl_spark.log` manda los mensajes de Spark a un archivo; `tee` muestra el resultado y lo guarda:

```
spark-submit ejemplos/ml/etl_spark.py 2> etl_spark.log | tee etl_spark.txt
```

Deja la tabla limpia en Parquet, en HDFS:

```
hdfs dfs -ls /user/root/ml/ventas_limpias
```

¿Dan lo mismo? Se comparan los dos resúmenes:

```
python3 ejemplos/ml/etl_pandas.py > etl_pandas.txt
diff <(sed -n '/=== RESUMEN/,/^$/p' etl_pandas.txt) <(sed -n '/=== RESUMEN/,/^$/p'
etl_spark.txt) \
  && echo "pandas y Spark dan lo mismo"
```

Desde un notebook, la tabla de Spark:

```
from pyspark.sql import SparkSession, functions as F

spark = SparkSession.builder.appName("mirar el ETL").getOrCreate()
limpias = spark.read.parquet("hdfs:///user/root/ml/ventas_limpias")
limpias.printSchema()
(limpias.groupBy("producto")
        .agg(F.sum("importe").alias("facturado"))
        .orderBy(F.desc("facturado"))
        .show())
```

`explain()` muestra el plan que arma Spark antes de correr (lee sólo las columnas que usa):

```
limpias.filter(F.col("producto") == "yerba").groupBy("sucursal").count().explain()
```

Y fila por fila contra la de pandas (`toPandas()` trae todo a la memoria del notebook: sólo con tablas que entren):

```
import pandas as pd

de_spark = limpias.toPandas().sort_values("ticket").reset_index(drop=True)
de_pandas = pd.read_parquet("ventas_limpias.parquet")
print(len(de_spark), len(de_pandas))
print((de_spark["ticket"].values == de_pandas["ticket"].values).all(),
      (de_spark["importe"].values == de_pandas["importe"].values).all())
spark.stop()
```

6. Modelos con scikit-learn

Para aprender, conviene empezar con datos donde **se sabe la respuesta**. `generar_senal.py` arma 20.000 clientes con reglas escritas en el propio script:

Columna	Qué es
`antiguedad_meses`, `visitas_mes`, `ticket_promedio`, `reclamos`	Las entradas
`edad`	**No influye en nada**: está para ver si los modelos se dan cuenta
`segmento_real`	ocasional / frecuente / mayorista: la respuesta del clustering, **no** para entrenar
`gasto_mes_siguiente`	Para regresión: `5000 + 1500 × visitas_mes + 0,8 × ticket_promedio` + ruido
`vuelve`	Para clasificación: 1 si vuelve a comprar el mes que viene

```
python3 ejemplos/ml/generar_senal.py
```

En un notebook, a mirar:

```
import pandas as pd

clientes = pd.read_csv("clientes_senal.csv")
clientes.describe().round(1)
```

```
clientes.groupby("segmento_real")[["visitas_mes", "ticket_promedio",
"gasto_mes_siguiente"]].mean().round(1)
```

La regla real de `vuelve`, escrita a mano (acierta casi siempre: el resto es el ruido):

```
regla = ((clientes["visitas_mes"] >= 4) & (clientes["reclamos"] <= 1)) |
(clientes["antiguedad_meses"] >= 36)
print((regla.astype(int) == clientes["vuelve"]).mean())
pd.crosstab(regla, clientes["vuelve"])
```

`modelos_sklearn.py` entrena, sobre el 80 % y probando con el 20 % que no vio:

1. **Regresión lineal** para `gasto_mes_siguiente`: muestra los coeficientes aprendidos al lado de los de la fórmula.
2. **Clasificación** de `vuelve` con un árbol de decisión, regresión logística y una red neuronal (`MLPClassifier`): muestra las reglas del árbol al lado de la regla real.
3. **Clustering** con KMeans (no supervisado): busca grupos sin que nadie le diga cuáles son y los compara con `segmento_real`.

De cada uno: aciertos, validación cruzada, cuánto tarda en entrenar y cuánta memoria usa. Deja `metricas_sklearn.json` y `clusters_sklearn.png`.

```
python3 ejemplos/ml/modelos_sklearn.py
```

El árbol encuentra los cortes de la regla real: `antiguedad_meses <= 35.50` y `visitas_mes <= 3.50`. Y no usa `edad`. El gráfico de los grupos:

```
from IPython.display import Image
Image("clusters_sklearn.png")
```

Paso a paso en un notebook

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

ENTRADAS = ["antiguedad_meses", "visitas_mes", "ticket_promedio", "reclamos", "edad"]
X, y = clientes[ENTRADAS], clientes["vuelve"]
X_ent, X_prueba, y_ent, y_prueba = train_test_split(X, y, test_size=0.2, random_state=42)
arbol = DecisionTreeClassifier(max_depth=3, random_state=42).fit(X_ent, y_ent)

nuevos = pd.DataFrame({"antiguedad_meses": [40, 10, 10], "visitas_mes": [1, 6, 6],
                       "ticket_promedio": [5000, 9000, 9000], "reclamos": [0, 0, 3],
                       "edad": [30, 30, 30]})
print(arbol.predict(nuevos))
print(arbol.predict_proba(nuevos).round(2))
```

La matriz de confusión y las métricas por clase:

```
from sklearn.metrics import confusion_matrix, classification_report

prediccion = arbol.predict(X_prueba)
print(confusion_matrix(y_prueba, prediccion))
print(classification_report(y_prueba, prediccion, digits=3))
```

Sobreajuste: con más profundidad, el árbol acierta más en entrenamiento pero no en prueba:

```
for profundidad in [1, 2, 3, 5, 10, None]:
    a = DecisionTreeClassifier(max_depth=profundidad, random_state=42).fit(X_ent, y_ent)
    print(f"max_depth={profundidad}: entrenamiento {a.score(X_ent, y_ent):.3f},"
          f" prueba {a.score(X_prueba, y_prueba):.3f}, hojas {a.get_n_leaves()}")
```

---

7. Los mismos modelos con Spark MLlib

scikit-learn entrena en la memoria de una compu; **Spark MLlib** reparte en el cluster. `modelos_spark.py` entrena los mismos modelos (regresión lineal, árbol, logística, red neuronal `MultilayerPerceptron`, validación cruzada para elegir la profundidad y KMeans), cada uno al lado de la regla real, con lo que tarda. Lee de HDFS:

```
hdfs dfs -mkdir -p /user/root/ml
hdfs dfs -put -f clientes_senal.csv /user/root/ml/
hdfs dfs -ls /user/root/ml
```

```
spark-submit ejemplos/ml/modelos_spark.py 2> modelos_spark.log | tee modelos_spark.txt
```

Las dos librerías lado a lado, en un notebook:

```
import json
import pandas as pd

sk = json.load(open("metricas_sklearn.json"))
ml = json.load(open("metricas_spark.json"))
pd.DataFrame({
    "scikit-learn": {n: m["aciertos"] for n, m in sk["clasificacion"].items()},
    "Spark MLlib": {n: m["aciertos"] for n, m in ml["clasificacion"].items()},
}).round(3)
```

Con datos de este tamaño, scikit-learn es más rápido: Spark paga el costo de repartir. Spark gana cuando los datos no entran en una compu.

Paso a paso: entradas en un vector, estimador y modelo

En MLlib las entradas van juntas en una columna-vector (`VectorAssembler`):

```
from pyspark.sql import SparkSession
from pyspark.ml.feature import VectorAssembler

spark = SparkSession.builder.appName("MLlib paso a paso").getOrCreate()
ENTRADAS = ["antiguedad_meses", "visitas_mes", "ticket_promedio", "reclamos", "edad"]
clientes = spark.read.csv("hdfs:///user/root/ml/clientes_senal.csv", header=True,
inferSchema=True)
datos = VectorAssembler(inputCols=ENTRADAS, outputCol="features").transform(clientes)
datos.select("features", "vuelve").show(3, truncate=False)
```

Un **estimador** (`DecisionTreeClassifier`) con `fit` da un **modelo** (`DecisionTreeClassificationModel`), que con `transform` agrega la predicción:

```
from pyspark.ml.classification import DecisionTreeClassifier

ent, prueba = datos.randomSplit([0.8, 0.2], seed=42)
estimador = DecisionTreeClassifier(labelCol="vuelve", maxDepth=3, seed=42)
modelo = estimador.fit(ent)
print(type(estimador).__name__, "->", type(modelo).__name__)
modelo.transform(prueba).select("features", "vuelve", "probability", "prediction").show(5,
truncate=False)
```

```
print(modelo.featureImportances)
spark.stop()
```

`edad` (la quinta) tiene importancia 0 o casi.

---

8. De MongoDB a pandas

Necesita `MONGODB=si` (el `docker run` de esta guía ya lo trae):

```
mongosh --quiet tienda --eval "db.pedidos.countDocuments()"
```

`mongo_pandas.py` lee los 20.000 pedidos con **pymongo** y los aplana en tablas de pandas: `pd.json_normalize` convierte cada subdocumento en columnas (`entrega.tipo`, `cupon.codigo`…) y los campos que un pedido no tiene quedan como `NaN`. Después calcula lo mismo con pandas y con el `aggregate` de MongoDB, y compara:

```
python3 ejemplos/ml/mongo_pandas.py
```

Los ítems de cada pedido (una lista adentro del documento) van a otra tabla, una fila por ítem (fragmento del script):

```
items = pd.json_normalize(pedidos, record_path="items", meta=["_id", "canal", ["entrega",
"tipo"]])
```

En un notebook:

```
import pandas as pd
from pymongo import MongoClient

db = MongoClient("mongodb://localhost:27017")["tienda"]
pedidos = list(db.pedidos.find({}, {"items": 0}))
tabla = pd.json_normalize(pedidos)
tabla.dtypes
```

```
tabla.groupby("canal")["total"].agg(["count", "sum"])
```

```
tabla.groupby("entrega.tipo")["total"].mean().round(0)
```

---

9. El estudio de ML desde JupyterLab

`sandbox_ml` es una librería propia del sandbox (Python puro, en `/opt/sandbox/ml/sandbox_ml`, se importa desde cualquier `python3`). Es la que usa la pantalla de ML de la versión `full`; en `ml` se usa desde un notebook. Arma experimentos reproducibles:

Módulo	Qué hace
`catalogo`	Los modelos disponibles (scikit-learn, XGBoost y Spark MLlib) con sus hiperparámetros, métricas y validaciones
`receta`	Un experimento entero en un JSON: datos, tarea, columnas, limpieza, modelo. `receta.validar(...)` lo revisa
`codigo`, `codigo_spark`	Receta → un script de Python (scikit-learn) o PySpark (MLlib) completo, que se puede leer y correr solo
`limpieza`, `transformadores(_spark)`	La limpieza columna por columna, adentro del Pipeline
`fuentes`, `perfil`, `perfil_spark`	Leer cualquier formato de HDFS o una tabla de Hive, y su perfil (nulos, tipos, distribución)
`estudio`	El registro de experimentos, en `/hadoop/ml/experimentos` (en el volumen)
`biblioteca`	Los modelos elegidos, con versiones y estado (producción…)
`lote`, `deriva`, `predecir`, `exportar`	Predecir por lotes, medir si los datos nuevos se parecen a los de entrenamiento, predecir filas sueltas, exportar un modelo a un `.zip`
`generar`	Un trabajo de Spark que genera datasets sintéticos del tamaño que se pida, con señal y suciedad

Un recorrido completo, con un millón de clientes. Los bloques corren en orden; necesitan el cluster con `SERVICIOS=todo` y memoria libre en YARN (cerrá otros notebooks con Spark).

Paso 1. Generar el dataset en HDFS, desde la terminal:

```
spark-submit --master yarn /opt/sandbox/ml/sandbox_ml/generar.py clientes \
  --filas 1000000 --salida /user/root/ml/generado/clientes_1m --suciedad 0.05
```

Paso 2. La receta y su script, desde un notebook:

```
from sandbox_ml import codigo, receta

r = receta.validar({
    "datos": {"origen": "hdfs", "ruta": "/user/root/ml/generado/clientes_1m", "formato":
"parquet"},
    "tarea": "clasificacion", "objetivo": "vuelve",
    "columnas": {"cliente_id": "id", "antiguedad_meses": "numerica", "visitas_mes":
"numerica",
                 "ticket_promedio": "numerica", "reclamos": "numerica", "edad": "numerica",
                 "segmento_real": "categorica", "gasto_mes_siguiente": "ignorar", "anio":
"ignorar", "mes": "ignorar"},
    "modelo": {"id": "logistica_spark"},
    "limpieza": {"pasos": [{"tipo": "nulos", "columna": "edad", "estrategia": "mediana"},
                           {"tipo": "texto", "columna": "segmento_real", "ops": ["recortar",
"minusculas"]}]},
})
open("/datos/churn.py", "w").write(codigo.generar(r))
```

`churn.py` queda en tu carpeta `datos/`: es un script normal, se puede abrir y leer.

Paso 3. Correrlo en el cluster:

```
spark-submit --master yarn --num-executors 2 --executor-memory 1g /datos/churn.py
```

Paso 4. La tabla de posiciones y la biblioteca:

```
from sandbox_ml import estudio
estudio.tabla("clasificacion")       # la tabla de posiciones, como DataFrame
```

```
from sandbox_ml import biblioteca

exp = estudio.experimentos()[0]["id"]          # el más nuevo
biblioteca.registrar("churn", f"/hadoop/ml/experimentos/{exp}", nota="logística con 1 millón
de clientes")
biblioteca.promover("churn", 1, nota="primera en producción")
biblioteca.listar()
```

Paso 5. Predecir por lotes con la versión en producción:

```
from pathlib import Path
from sandbox_ml import lote

m, x = biblioteca.elegir("churn")              # la versión en producción
datos = {"origen": "hdfs", "ruta": "/user/root/ml/generado/clientes_1m", "formato":
"parquet"}
salida, _ = lote.normalizar_salida({"ruta": "/user/root/ml/predicciones/churn"}, "churn",
x["version"])
Path("/datos/lote.py").write_text(lote.generar(x, biblioteca.carpeta_version("churn",
x["version"]), datos, salida, nombre="churn"))
```

```
spark-submit --master yarn /datos/lote.py | grep -E "deriva de|predicciones en"
```

Lo mismo con pantallas (elegir datos, modelo, hiperparámetros, ver la tabla de posiciones) es la versión `full`.

---

10. Problemas comunes y limpieza

```
docker exec bigdata estado
```

Si JupyterLab no aparece en `estado` o el navegador no abre el 8888, su log:

```
docker exec bigdata tail -20 /var/log/jupyter.log
```

Síntoma	Causa y arreglo
http://localhost:8888 no abre	Faltó `-p 127.0.0.1:8888:8888` en el `docker run`, o `JUPYTER=no`. Recrear el contenedor
`estado` dice `Version de la imagen: base`	El contenedor es de la imagen `base`: `docker rm -f bigdata` y el `docker run` con `:ml`
`ModuleNotFoundError: No module named 'pandas'`	Mismo caso: estás en `base`
El log dice que `JUPYTER` no aplica	Se pidió `JUPYTER=si` en una imagen sin JupyterLab (`base`)
Un `spark-submit` queda en `ACCEPTED`	YARN sin memoria: una sesión de Spark de un notebook sigue abierta. `spark.stop()` o *Kernel → Shut Down Kernel*
El notebook se reinicia solo (*Kernel died*)	Sin memoria: `toPandas()` de una tabla grande o poca RAM para Docker
`etl_spark.py` o `modelos_spark.py` dicen `Path does not exist`	Faltó el `hdfs dfs -put` de su parte
`mongo_pandas.py` dice `Connection refused`	MongoDB apagado: recrear con `-e MONGODB=si`
Hace falta otra librería	`pip install <librería>` adentro (o `%pip install` en una celda). Se pierde al recrear el contenedor

Para repetir la guía desde cero (borra lo que generó, no toca lo demás):

```
hdfs dfs -rm -r -f /user/root/ml
rm -f ventas_sucias.csv ventas_limpias.parquet clientes_senal.csv \
      metricas_sklearn.json metricas_spark.json clusters_sklearn.png
```